

τ_0 -VLA: A Hierarchical VLA Foundation Model with World-Model-Guided Test-Time Computation

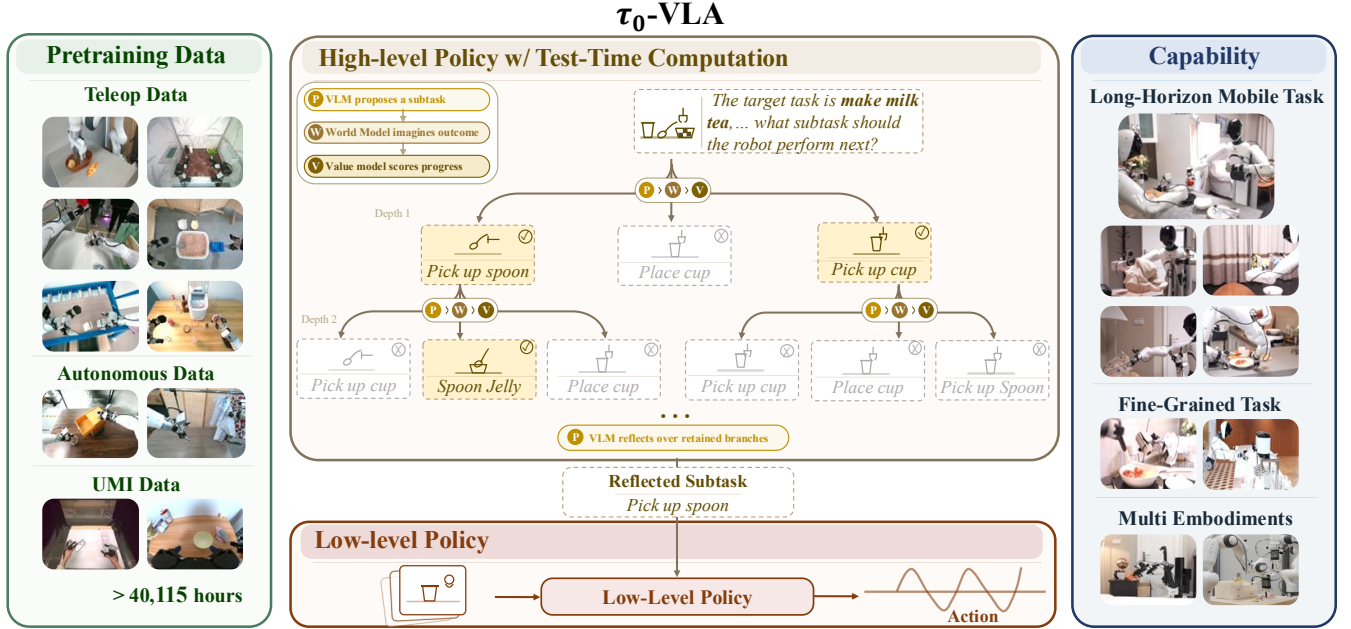


Fig. 1. **Overview of τ_0 -VLA.** A memory-augmented high-level planner uses world-model-guided test-time computation to search over subtask sequences. At each expansion, a VLM proposes candidates, a world model predicts their visual outcomes, and a value model scores task progress; beam search retains promising branches, and reflection commits the next subtask. The committed subtask conditions a low-level whole-body policy trained on approximately 40,115 hours of heterogeneous robot data, supporting long-horizon mobile manipulation, fine-grained task execution, and deployment across multiple embodiments.

Abstract—Long-horizon mobile manipulation requires robots to maintain task progress, reason about future consequences, and execute reliable whole-body actions over extended interactions. Existing vision-language-action (VLA) models often struggle to coordinate these capabilities, allowing local errors to accumulate into repeated actions, skipped stages, and goal drift. We introduce τ_0 -VLA, a hierarchical VLA system composed of a High-Level Planner and a low-level whole-body policy. The planner maintains execution memory and proposes the next subtask. When a decision requires additional deliberation, it allocates test-time computation to generate candidate subtasks, predict their visual consequences with a world model, evaluate task progress, and select the command passed to the low-level policy. This conditional mechanism concentrates additional computation on consequential decisions while preserving efficient execution on routine steps. The low-level policy operates in a unified 40-dimensional action space and is trained on approximately 40,115 hours of heterogeneous robot demonstrations and multimodal data across multiple embodiments. Across closed-loop evaluations on multiple long-horizon manipulation tasks, hierarchical test-time computation improves task success by an average of xxxx percentage success rates over whole-task inference and other baseline methods.

I. INTRODUCTION

Many robotic tasks that matter in practice unfold over long horizons. Cleaning a room, cooking a dish, or preparing a drink may require many subtasks to be completed in a coherent order from minutes to hours. The robot must locate objects, interact with articulated structures, verify outcomes, and recover from local failures. Such tasks are not merely long sequences of motor commands, but sequences of consequential decisions. At each high-level decision point, the robot must infer what has been accomplished, determine what remains, and decide what to do next. An error at this level is not generally repaired by more accurate low-level motor control: the robot may execute the wrong subtask perfectly.

Vision-language-action (VLA) models provide a powerful foundation for this setting by combining semantic task understanding with continuous robotic control [7, 38, 20, 25, 5, 27]. Hierarchical policies and dual-system VLAs have extended this paradigm to multi-stage tasks by separating high-level semantic decisions from low-level action generation [19, 4, 33, 24, 13, 16, 27, 28]. While subtask decomposition is an effective interface for long-horizon manipulation, selecting the

next subtask often remains an amortized prediction under a fixed inference budget. Even when conditioned on execution history and deployed in a closed loop, this prediction does not necessarily compare candidate subtasks through the physical states they are expected to produce. Replanning then responds only after execution, when the cost of a poor commitment may already have been incurred. We therefore allocate test-time computation to high-level reasoning, allowing the policy to spend more computation on ambiguous decisions and compare alternatives through their predicted downstream consequences before acting.

Action-level sampling and verification are well suited to resolving local control ambiguities, but may not expose consequences at the task level [23, 21, 32]. Language-only reasoning reaches farther, but may compare alternatives without grounding them in the physical states they are expected to produce. Generative models of future observations provide such grounding [10, 6, 11, 12]. Subtask-level decisions offer a useful granularity for this computation: they are sparse enough to support more expensive reasoning, temporally extended enough to produce meaningful state changes, and sufficiently bounded to be executed and evaluated in the physical world.

We present τ_0 -VLA, a dual-system VLA that scales test-time reasoning over high-level subtask decisions. At each decision point, a memory-augmented propose model generates a set of open-ended candidate subtasks. A subtask-conditioned visual outcome model predicts the terminal observation induced by each candidate, and a separate value model scores the predicted state according to task progress, instruction consistency, and execution history. Promising candidates are recursively expanded through subtask-level beam search, and a reflective model commits to the immediate next subtask after comparing the resulting branches. The branching factor and search depth directly control the inference budget, while a learned routing mechanism invokes the expensive search-and-reflect pathway only for difficult decisions. Only the selected subtask is executed; the robot then observes the actual outcome, updates its memory, and replans. This turns next-subtask prediction into a compute-scalable, consequence-aware reasoning process while preserving closed-loop execution.

Effective reasoning also requires an estimate of task progress that remains consistent with the physical scene. We therefore represent execution history with a compact but revisable memory. Using existing robot demonstrations and hierarchical annotations, we automatically construct structured supervision for reasoning, memory maintenance, and next-subtask prediction without additional manual labeling. We perturb the input memory while retaining the corrected target, creating examples in which the planner must advance a lagging memory, retract an over-optimistic one, or recover from failed execution. The visual outcome model is trained from subtask-aligned start–end frame pairs, providing transition supervision at the same temporal resolution as the high-level planner.

The selected subtask is grounded by a generalist low-level VLA that couples a pretrained vision-language backbone

with a Mixture-of-Transformers action head. A unified 40-dimensional state and action representation spans the end effectors, arm joints, grippers, waist, and mobile base, enabling fixed-base, bimanual, and mobile whole-body control across multiple robot embodiments. We train this policy on approximately 40,115 hours of heterogeneous robot demonstrations, together with multimodal co-training data. The resulting actor provides a common language-conditioned execution interface without requiring a separate controller for each subtask. We evaluate the complete system on long-horizon real-world tasks involving object search, navigation, articulated-object interaction, tool use, progress tracking, and recovery.

Our contributions are threefold:

- We introduce compute-scalable high-level reasoning for hierarchical VLAs, combining open-ended subtask proposal, subtask-conditioned visual outcome prediction, value-guided multi-step search, reflective commitment, and closed-loop execution.
- We introduce a correctable execution memory and an automatic data-construction procedure for training progress tracking, memory correction, and failure recovery from existing robot demonstrations.
- We couple the high-level system to a unified whole-body VLA trained on approximately 40,115 hours of heterogeneous robot data, and evaluate the complete system on long-horizon tasks across multiple robot embodiments.

II. RELATED WORK

Long-horizon robot systems must combine a broadly capable control policy, temporal abstraction, and a mechanism for allocating inference-time computation. We organize related work around these three design requirements: VLA policies, hierarchical robot policies, and world-model-guided test-time computation.

A. Vision-Language-Action Models

Vision-language-action (VLA) models condition robot control on visual observations and language instructions, typically by adapting a pretrained vision-language backbone to output actions. RT-1 [7] and RT-2 [38] demonstrated that large-scale semantic knowledge can transfer to robot control, while OpenVLA [20] and Octo [25] broadened this recipe across embodiments. More recent generalist VLAs target continuous control, cross-embodiment transfer, and open-world generalization, including π_0 [5], $\pi_{0.5}$ [27], $\pi_{0.7}$ [28], GR00T N1.7 [24], X-VLA [36], LingBot-VLA [2], and Gemini Robotics 1.5 [16]; SmolVLA [34] instead emphasizes efficient deployment. These policies provide strong short-horizon control, but long-horizon execution also requires explicit progress tracking and subtask selection. τ_0 -VLA integrates a whole-body low-level policy with a High-Level Planner that manages these longer-timescale decisions.

B. Hierarchical Robot Policies

Hierarchical policies separate temporally extended decisions from continuous control. SayCan [19] grounds language-

model decisions in robot affordances, while RT-H [4] introduces language-based action hierarchies within a single policy. Recent systems instantiate this separation in different ways. Hi Robot [33] maps open-ended instructions and user feedback to atomic commands for a low-level VLA, whereas Helix [13] passes latent semantic representations from a slower VLM to a fast visuomotor policy. GR00T N1.7 [24] uses an action cascade comprising a reasoning VLM and a diffusion-transformer controller, while Gemini Robotics 1.5 [16] combines an embodied-reasoning orchestrator with a VLA that can interleave language-based reasoning and physical action. $\pi_{0.5}$ [27] performs two-stage hierarchical inference within a shared VLA model, first predicting a semantic subtask and then generating actions conditioned on it; $\pi_{0.7}$ [28] further conditions control on commands from a learned high-level language policy and subgoal images generated by a world model.

In τ_0 -VLA, hierarchy is not only an interface between semantic and motor timescales. The High-Level Planner maintains explicit execution memory and conditionally invokes world-model-guided test-time computation to expand candidate subtasks, predict their visual consequences, assess task progress, and reflect before commitment. This design allocates additional computation selectively at consequential subtask boundaries.

C. World Models and Test-Time Computation

World models predict future states under candidate actions and support planning and policy learning [17, 18]. In robot manipulation, UniPi [10] generates goal-directed videos that an inverse-dynamics model decodes into actions, SuSIE [6] synthesizes subgoal images for a low-level controller, and GR-2 [9] and WorldVLA [8] jointly model future observations and actions. Video-Language-Planning [11] performs tree search with a vision-language policy and value function plus a text-to-video dynamics model. Reflective Planning [12] imagines future states and revises multi-stage manipulation plans. At the action level, V-GPS [23], RoboMonkey [21], and RoVer [32] allocate additional inference to generate-and-verify or value-steered selection. Our High-Level Planner applies this idea at the subtask boundary. It predicts the visual consequences of candidate language subtasks, scores their consistency with task progress, and reflects before physical execution. The extra computation therefore evaluates temporally extended decisions while the low-level policy retains a stable whole-body control interface.

III. SYSTEM OVERVIEW

A. Problem Formulation

We consider a long-horizon manipulation task specified by a language instruction ℓ and a sequence of robot observations $\{o_t\}_{t=1}^T$. In hierarchical TTC execution, τ_0 -VLA decomposes the task into language subtasks z_k and corresponding low-level action segments τ_k :

$$g = (z_1, \tau_1, z_2, \tau_2, \dots, z_K, \tau_K). \quad (1)$$

Subtask z_k remains active over the interval $[t_k, t_{k+1})$. During this interval, the low-level policy predicts short action chunks:

$$a_{t:t+H} = \pi_\theta(o_t, c_t), \quad t_k \leq t < t_{k+1}, \quad (2)$$

where c_t denotes the language command supplied to the policy. In direct execution, the high-level planner is bypassed and $c_t = \ell$ throughout the task. In hierarchical TTC execution, $c_t = z_k^*$ over $[t_k, t_{k+1})$, where the planner selects z_k^* from the current observation and execution history.

B. Architecture

τ_0 -VLA comprises two components operating at different timescales. The *high-level planner* operates at subtask boundaries, maintaining execution memory, deciding when to invoke additional test-time computation, and selecting the next subtask. The *low-level policy* operates at the faster control timescale, mapping the current observation and language command to whole-body robot actions. This separation enables long-horizon progress tracking and consequence-aware planning without modifying the low-level control interface. Figure 2 provides an overview, and Section IV describes training and inference.

C. High-Level Planner

The high-level planner comprises a proposal model P , a visual world model $\mathcal{W}$, a value model V , and a reflective model F . The proposal, value, and reflective models all share the same Qwen3.5-VL-9B backbone [29], while the world model is instantiated from Step1X-Edit [22]. Together, these modules propose, predict, evaluate, and select the robot’s next subtask.

Proposal and execution memory. At subtask boundary k , the planner conditions on

$$h_k = (\ell, \varsigma, \mathcal{M}_k, o_{t_k}), \quad (3)$$

where $\mathcal{M}_k$ records completed stages and relevant details of the current execution state, and ς is an optional user-provided stage-level plan (an ordered L2 sequence). When present, ς conveys the user’s preferred decomposition and ordering—for example, cutting tomatoes before or after beating eggs—which the proposal model follows while staying grounded in the visual state; when absent, the planner decomposes the task autonomously. This conditioning is part of the model’s native input format and is present throughout training, so steering the same instruction toward a different valid ordering requires no weight updates. The proposal model emits structured `<think>`, `<memory>`, and `<subtask>` fields. The `<memory>` field produces the updated memory $\mathcal{M}_{k+1}$, making task progress explicit across long executions. For routine decisions, the model directly proposes the next subtask. When additional deliberation is needed, it emits a special `<DeepThink>` token that invokes the TTC route described in Section IV-D.

Consequence prediction and assessment. On the TTC route, P generates a set of candidate subtasks. For each candidate, $\mathcal{W}$ predicts its visual consequence, and V evaluates

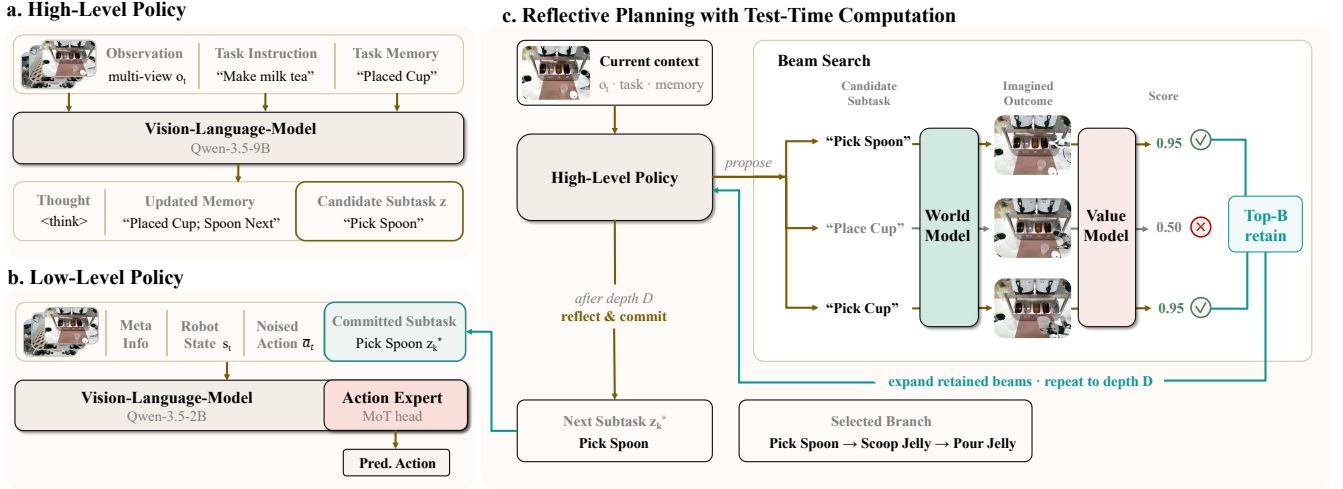


Fig. 2. **The τ_0 -VLA architecture.** (a) The High-Level Planner conditions on multi-view observations, the task instruction, and task memory to generate structured reasoning, an updated memory, and candidate subtasks. (b) The low-level policy conditions on the committed subtask together with observations and robot state; a vision-language backbone and Mixture-of-Transformers (MoT) action expert predict whole-body actions. (c) During world-model-guided test-time computation, the High-Level Planner proposes candidate subtasks, the world model predicts their visual outcomes, and the value model scores task progress. Beam search retains the top- B branches and recursively expands them to depth D , after which reflection commits the next subtask for low-level execution.

whether the predicted outcome advances the task consistently with the execution memory. The reflective model F then compares the candidate–outcome pairs and selects the next subtask. This separation of proposal, prediction, evaluation, and selection allows the planner to reject locally plausible actions whose predicted consequences conflict with observed task progress.

D. Low-Level Policy

The low-level policy uses a Qwen3.5-2B vision-language backbone [29] with a Mixture-of-Transformers (MoT) action head. At each full-attention layer, action-head tokens interact with the backbone tokens through joint attention while retaining separate parameters. The policy encodes multi-view observations, proprioceptive state, and the current language command, and then decodes a short action chunk.

Unified action space. To support heterogeneous embodiments with a single policy, we use unified 40-dimensional state and action representations. Fixed slots represent end-effector, arm-joint, gripper, waist, and mobile-base degrees of freedom. Each embodiment maps its available states and controls to these slots and masks unused dimensions. The resulting interface supports both fixed-base manipulation and coordinated whole-body mobile manipulation.

Language interface. The low-level architecture remains identical across the two execution modes; only the language input changes. In direct execution, the policy receives the original task instruction ℓ . In hierarchical TTC execution, it receives the bounded subtask z_k^* and executes under that command until the next planner invocation, after which the resulting observation is returned to the planner. This controlled comparison isolates the effects of hierarchical decomposition

and test-time computation from changes to the low-level policy.

IV. TRAINING AND SYSTEM INFERENCE

A. Data Sources

Low-level data. The low-level policy is trained on approximately 40,115 hours of real-world demonstrations spanning fixed-base, mobile, and bimanual embodiments. We combine about 32.7K hours of internally collected data with public robot datasets and multimodal co-training data that preserve the backbone’s vision-language capabilities during action learning. Appendix A-B details the embodiments, public sources, and multimodal mixture.

High-Level Planner data. Planner supervision is produced automatically from the dataset’s existing task annotations and segmented demonstrations, without additional per-sample human labeling. The annotations follow three semantic levels: a task instruction, an ordered stage description, and an executable subtask, denoted L1, L2, and L3 in our data pipeline. Conditioned on these annotations and videos, a multimodal model (google/Gemma4-31B-it) pre-labels per-step reasoning (<think>) and running progress memory (<memory>). Together with the existing <subtask> annotation, these fields form a unified training target. Multi-view keyframes at subtask boundaries turn the annotations into VQA samples for task decomposition, progress tracking, next-subtask prediction, error detection, and recovery. We also perturb the input memory to construct *transition*, *catch-up*, *rollback*, and *error-think* instances that target deployment-time memory drift. Although this supervision is generated automatically, the prompts and filters were developed under stratified human spot-checks that surfaced failure modes—most notably cross-

task contamination—which were then hardened into programmatic filters applied at scale; this pipeline removes 11.74% of structurally corrupted episodes, retains 40.4M (88.26%) clean samples, and leaves zero residual cross-task contamination after a 0.42% discard. The pipeline scales with the demonstration corpus and shares the same $\sim 40\text{K}$ -hour foundation as the low-level policy; Appendix A-C provides the construction details.

The two components use this foundation differently. The low-level policy learns directly from robot demonstrations and the multimodal mixture. The High-Level Planner shares the multimodal data, while its language modules and world model receive supervision derived from subtask annotations and start–end frame pairs.

B. Low-Level Policy Training

The low-level policy maps the current observation and a language-conditioned subtask to robot actions in the unified action space. We organize training into two phases. Pretraining builds a general robot policy from broad multimodal and robot-action data through Stage I and Stage II. Posttraining then adapts the policy to downstream deployment tasks through Stage III.

Stage I: vision-language and action co-training. We jointly train on multimodal and robot-action data. Recall that the MoT action head reads from the backbone through joint attention while keeping its own weights, which lets us update the two at different rates: we optimize both the backbone and the action head, but stop gradients from the action head at the backbone interface. This knowledge-isolation (KI) [26] schedule lets the action head learn control-relevant representations without rapidly overwriting the pretrained vision-language features, while co-training on multimodal data preserves the backbone’s native vision-language capability and mitigates early capability collapse.

Stage II: end-to-end action adaptation. We then enable end-to-end optimization of the full model for action learning. Multimodal data remain in the training mixture as an auxiliary regularization signal. In practice, action-only training substantially degrades vision-language capability and destabilizes optimization. Retaining multimodal supervision helps preserve generalization while allowing the full model to adapt to robot control.

Stage III: task-specific fine-tuning. For deployment on a target task, we further fine-tune the policy on a small set of task-specific demonstrations. This adapts the shared policy to the exact objects, viewpoints, and success criteria of each downstream task while retaining the broad capabilities acquired in the previous stages.

C. High-Level Planner Training

The High-Level Planner must generate the next subtask and assess its consequences before the robot acts. Its proposal, value, and reflective modules share a vision-language backbone and common pretraining stage, then specialize through role-specific fine-tuning. The visual world model is trained separately.

1) Pretraining: Before role-specific fine-tuning, the three language modules share a common pretraining stage. From ordinary robot demonstrations we automatically construct two forms of supervision without additional annotation: (i) **full reasoning**, which maps the current observation, the host-task instruction, and the running memory to the structured `<think>`, `<memory>`, and `<subtask>` fields; and (ii) **next-subtask prediction**, which predicts the immediate next subtask from the instruction and observation alone. Training on these supervisions together with the shared multimodal corpus equips the backbone with structured task decomposition, memory summarization, and progress tracking. The subsequent fine-tuning stages specialize this shared backbone into the propose, value, and reflective models.

2) Role-Specific Fine-Tuning: Proposal model. The proposal model generates candidate subtasks while maintaining and updating the task memory. Pretraining provides the basic capability; fine-tuning strengthens it against the memory drift that arises during real deployment. Demonstrations contain only memory consistent with the visual state, but at deployment the running memory may lag, run ahead, or misrepresent progress after an unnoticed failure. To close this gap without additional annotation, we synthesize training instances by perturbing the input memory of existing demonstrations while keeping the corrected target unchanged. This yields five instance families: *within-subtask* steps with already-aligned memory; *transition* steps at segment boundaries; *catch-up* steps where a lagging memory must be advanced; *rollback* steps where an over-optimistic memory must be retracted; and *error-think* steps from annotated failure frames, where the `<think>` field must flag the failure before the memory is repaired. For error-think steps, recoverable failures (e.g. an empty grasp) are retried in place, whereas failures that undo prior progress (e.g. a dropped object) trigger a rollback to the preceding subtask. This annotation-free procedure teaches the model to recover the correct memory and next subtask rather than trusting a flawed history.

Value model. Built on the shared VLM backbone, the value model is cast as a multiple-choice VQA task: conditioned on the L1 host task, it inspects the world model’s imagined future state for a candidate subtask and predicts one of five ordinal quality levels—from *clearly wrong* to *clearly correct*—that map to scalar values in $[0.05, 0.95]$. Its training pairs each sampled state with a candidate subtask drawn from the proposal model’s own rollouts—multi-step plans generated offline by rolling forward over the world model’s predicted future states. Graded against the ground-truth next step, this data aligns training with the distribution of candidates the value model must score at deployment, where these values are used to rank competing subtasks during search.

Reflective model. Also sharing the VLM backbone, the reflective model receives the current observation, task context, memory, a candidate multi-step plan, and the plan’s predicted future state to output the correct immediate next subtask. During training, candidate plans are sourced from the same offline rollouts described above, paired with the ground-truth

Algorithm 1 Hierarchical TTC inference with the High-Level Planner

Require: Task instruction ℓ , proposal model P , world model $\mathcal{W}$, value model V , reflective model F , low-level policy π_θ , branching factor N , search depth D

- 1: Initialize execution memory $\mathcal{M}_1$
- 2: Observe o_{t_1}
- 3: **for** $k = 1, 2, \dots$ until the task termination criterion is met **do**
- 4: $h_k \leftarrow (\ell, \mathcal{M}_k, o_{t_k})$
- 5: $(q_k, \widehat{\mathcal{M}}_{k+1}) \leftarrow P(h_k)$
- 6: **if** $q_k \neq \text{<DeepThink>}$ **then**
- 7: $z_k^* \leftarrow q_k$ $\triangleright$ fast route
- 8: **else**
- 9: $\mathcal{Z}_k \leftarrow \text{PROPOSECANDIDATES}(P, h_k, N)$
- 10: $\mathcal{C}_k \leftarrow \text{SEARCH}(\mathcal{Z}_k, h_k, \mathcal{W}, V, P, D)$
- 11: $z_k^* \leftarrow F(h_k, \mathcal{C}_k)$ $\triangleright$ TTC route
- 12: **end if**
- 13: $(\tau_k, o_{t_{k+1}}) \leftarrow \text{EXECUTESUBTASK}(\pi_\theta, z_k^*)$
- 14: $\mathcal{M}_{k+1} \leftarrow \text{UPDATEMEMORY}(\widehat{\mathcal{M}}_{k+1}, z_k^*, \tau_k, o_{t_{k+1}})$
- 15: **end for**

next subtask as their target. By leveraging this shared data-generation strategy, the reflective model learns to correct the exact distribution of flawed plans it will encounter at inference. This pushes the model to ground its decisions in visual evidence and actively repair unreliable plans, rather than simply copying the first proposed action.

3) *World Model*: We train an image-editing world model on internally collected robot data. For each annotated subtask, we form start–end frame pairs from its temporal ranges: given a start frame and the subtask instruction, the model predicts the corresponding terminal frame,

$$\hat{o}_{\text{end}} = \mathcal{W}(o_{\text{start}}, z), \quad (4)$$

where o_{start} is the start observation, z is the subtask instruction, and $\hat{o}_{\text{end}}$ is the predicted terminal observation. This turns temporally grounded subtask annotations into large-scale image–text transition pairs. The resulting predictions give the High-Level Planner a visual preview of each candidate subtask, which the value and reflective models assess during search.

D. Hierarchical TTC Inference

At deployment, the High-Level Planner and low-level policy run in a closed loop, as summarized in Algorithm 1. At subtask step k , the proposal model reads the task instruction, current observation, and execution memory. If it emits a concrete subtask, the planner takes the fast route and immediately prompts the low-level policy. If it emits `<DeepThink>`, the planner allocates additional computation before physical execution.

On the TTC route, the proposal model generates a candidate set

$$\mathcal{Z}_k = \{z_k^1, \dots, z_k^N\} = P(h_k). \quad (5)$$

For every candidate, the world and value models predict and assess its consequence:

$$\hat{o}_k^i = \mathcal{W}(o_{t_k}, z_k^i), \quad v_k^i = V(h_k, z_k^i, \hat{o}_k^i). \quad (6)$$

Proposal, prediction, and scoring define one search level. For deeper search, each predicted outcome becomes an imagined observation from which the proposal model expands another level, with the imagined memory extended by the candidate and its outcome. Search recurses to depth D , and each first-level candidate receives an aggregate branch score s_k^i from its path.

The search produces branch summaries

$$\mathcal{C}_k = \{(z_k^i, \rho_k^i, \hat{o}_{k,D}^i, s_k^i)\}_{i=1}^N, \quad (7)$$

where ρ_k^i is the imagined subtask path and $\hat{o}_{k,D}^i$ its terminal predicted observation. The reflective model commits to the immediate next subtask,

$$z_k^* = F(h_k, \mathcal{C}_k), \quad (8)$$

and the low-level policy executes it in the physical environment. The resulting observation updates the execution memory before the next planner call. Thus, additional computation is conditional rather than paid at every subtask step. By contrast, τ_0 -VLA in direct-execution mode bypasses Algorithm 1 and conditions the same low-level policy directly on ℓ .

V. EXPERIMENTS

Our evaluation separates high-level decision quality, long-horizon system performance, and short-horizon multi-embodiment execution. At the high level, we evaluate next-subtask prediction, test-time computation, and the contribution of execution memory. We then evaluate the complete hierarchical system on long-horizon AGIBOT G1 tasks and separately test the shared low-level VLA through direct execution on shorter ARX and Franka tasks.

A. Evaluation Setup

Robot platforms. We evaluate on three embodiments: AGIBOT G1 [1] for the primary long-horizon tasks, ARX AC One [3] for cross-embodiment evaluation, and Franka Research 3 [14] for fixed-base instruction following and low-level control. All platforms provide multi-view RGB observations and proprioceptive state and map their controls into the unified action space. Hardware and sensing details are deferred to Appendix A-A.

Task suite. The evaluation covers five household tasks and one Franka benchmark comprising three separately evaluated instruction-following task groups. Clean Room, Ingredient Prep, and Tomato and Egg Stir Fry on AGIBOT G1, together with Collect Laundry on ARX AC One, require mobile manipulation. Make Milk Tea on AGIBOT G1 and all three Tidy Makeup Table groups on Franka are manipulation-only evaluations executed without base motion. Tidy Makeup Table serves as our instruction-following generalization benchmark.

- **Clean Room (AGIBOT G1; 21 steps).** The robot navigates through a bedroom and living room; collects



Fig. 3. **Illustration of Evaluation Tasks.** (a) Clean Room: the robot collects dirty clothes, hangs a handbag, hands over a blanket, and tidies the living-room table. (b) Ingredient Prep / Tomato and Egg Stir Fry: the preparation task retrieves tomatoes and eggs and cracks and mixes the eggs; the cooking task chops the tomatoes, moves the prepared ingredients beside the induction cooker, adds oil and the egg mixture, stir-fries the eggs, and adds the tomatoes. The two phases are evaluated as separate tasks. (c) Make Milk Tea: the robot dispenses a cup, adds toppings, pours milk and tea, seals the cup, and inserts a straw. (d) Collect Laundry (ARX AC One): the robot picks up clothes beside the bed, moves to a laundry basket, and places them inside. (e) Tidy Makeup Table (Franka) comprises three separately evaluated instruction-following groups—Cotton Pad, Eyelash Curler, and Makeup Puff—that require different actions from matched observations.

scattered dirty clothes in a laundry basket; hangs a handbag from an arbitrary initial location on a coat rack; finds a blanket and hands it to a person; and tidies the living-room table.

- **Ingredient Prep (AGIBOT G1; 18 steps).** The robot moves between the refrigerator and preparation counter; retrieves tomatoes and eggs; and cracks and mixes the eggs.
- **Tomato and Egg Stir Fry (AGIBOT G1; 13 steps).** The robot chops the tomatoes; transfers the prepared tomatoes and egg mixture to the induction cooker; turns it on; adds oil and the egg mixture; stir-fries the eggs; adds the tomatoes and salt; finishes cooking and plating; turns off the cooker; and carries the dish to the table. Repeated salt addition invalidates task success but receives 0.5 PAP credit for the salt-addition subtask.
- **Make Milk Tea (AGIBOT G1; 13 steps).** The robot places a cup at the preparation station; adds two toppings; pours milk and tea in sequence; attaches a lid; and inserts a straw. A complete rollout takes approximately

2 min 50 s.

- **Collect Laundry (ARX AC One; 5 steps).** The robot searches for and moves to the drawer cabinet; picks up the dirty garment from the bedside table with its left arm; closes the cabinet door with its right arm; searches for and moves to the laundry basket; and places the garment in the basket with its left arm. A trial succeeds if the garment remains supported by the basket without falling out at the end of execution. If the garment is only partially inside, the final placement milestone receives 0.5 rather than full PAP credit.
- **Tidy Makeup Table (Franka; three task groups).** This benchmark pairs matched visual states with instructions that require different target objects, acting arms, action sequences, and destinations. We score three groups independently: *Cotton Pad* (2 steps), in which the left arm picks up and places the cotton pad; *Eyelash Curler* (2 steps), in which the left arm picks up the eyelash curler and places it in its designated compartment; and *Makeup Puff* (4 steps), in which both arms open the drawer, the

left arm picks up the makeup puff from the tabletop and places it inside, and both arms close the drawer.

High-level planner evaluation. We first isolate high-level decision making from physical execution using annotated samples collected at subtask boundaries. Each sample contains the task instruction, current robot observation, execution memory when applicable, and the ground-truth next subtask. We report next-subtask prediction accuracy and inference time. For the open-loop TTC comparison, a common LLM judge determines whether each prediction is semantically consistent with the annotation. Within each comparison, all methods are evaluated on the same annotated samples. The memory ablation changes whether execution memory is provided, whereas Plan Once, Best-of- N , and TTC compare different decision-time inference procedures within the high-level planner. In particular, Plan Once still invokes the high-level planner and is distinct from direct-execution τ_0 -VLA in the overall system evaluation. The judging protocol is summarized in Appendix B-B.

Physical-robot evaluation metrics. We then evaluate complete systems through closed-loop execution on physical robots. We apply the same task-specific terminal conditions and milestone annotations to every method. We report *success rate* (SR), the fraction of trials that satisfy all terminal conditions by the end of execution, and *prerequisite-aware progress* (PAP), which measures valid partial completion. SR depends only on the final outcome: a trial remains successful even if some actions require multiple attempts, provided that all terminal conditions are eventually met. A trial requiring human assistance is terminated and counted as a failure. PAP additionally reflects execution inefficiencies. It represents each task as a partial-order graph of its original annotated subtasks and credits a subtask only when both its completion condition and all prerequisites are satisfied. A subtask completed on the first attempt receives full credit, one completed only after repeated attempts receives 0.5 credit, and an incomplete subtask receives no credit. We additionally assign 0.5 credit to predefined task-specific partial-completion states. Independent branches are scored separately: for example, failing to add the first topping in Make Milk Tea does not prevent credit for a correctly added second topping. When an execution record groups multiple subtasks, we resolve their outcomes at the original subtask granularity using the accompanying action-level annotations before applying prerequisite constraints. Only valid subtasks completed before human intervention contribute to PAP. We normalize the total subtask credit by the number of annotated subtasks and average it across trials. For the controlled comparison of our two variants, the observation and action interfaces and the low-level policy are fixed. Standalone τ_0 -VLA conditions the low-level policy on the full task instruction, whereas τ_0 -VLA w/ Hierarchical System supplies bounded subtasks from the high-level planner during execution.

B. Overall System Evaluation

Table I reports the four long-horizon AGIBOT G1 tasks: Clean Room, Ingredient Prep, Tomato and Egg Stir Fry, and

Make Milk Tea. These tasks require navigation, object search, manipulation, state tracking, and recovery across multiple stages.

Make Milk Tea results. On the 13-step Make Milk Tea task, τ_0 -VLA reaches 70% SR and 97.69% PAP, while τ_0 -VLA w/ Hierarchical System reaches 50% SR and 91.42% PAP. Both systems outperform the external baselines: $\pi_{0.5}$ reaches 10% SR and 79.23% PAP, while LingBot-VLA and GR00T reach 0% SR and 63.85% and 28.46% PAP, respectively. GR00T does not reach the topping stage, LingBot-VLA often skips topping pickup, and $\pi_{0.5}$ struggles with lid attachment and occasionally restarts after skipping steps. Remaining τ_0 -VLA failures occur at straw insertion, whereas failures with the hierarchical system concentrate on lid attachment.

Clean Room results. The hierarchical system reaches 8/10 SR and 97.86% PAP, compared with 4/10 SR and 91.43% PAP for standalone τ_0 -VLA. The hierarchy thus adds four successful trials and 6.43 PAP points while holding the low-level policy fixed. $\pi_{0.5}$ also reaches 4/10 SR but lower 87.14% PAP, whereas LingBot-VLA and GR00T complete no trial and reach 67.14% and 61.90% PAP, respectively. Their failures concentrate on handbag hanging and the later room-tidying stages. The improvement is consistent with explicit execution memory helping the system retain progress across room transitions and avoid losing track of late-stage subtasks.

Ingredient Prep results. The hierarchical system achieves 8/10 SR and 95.00% PAP, improving over standalone τ_0 -VLA by four successful trials and 26.67 PAP points. The corresponding results are 2/10 and 72.50% for $\pi_{0.5}$, 1/10 and 56.94% for GR00T, and 0/10 and 27.22% for LingBot-VLA. Most failures arise during egg pickup, cracking, or stirring. Because these actions are prerequisites for the later preparation stages, an early error also suppresses downstream PAP; maintaining the completed-stage memory is therefore especially valuable on this task.

Tomato and Egg Stir Fry results. The hierarchical system obtains 4/10 SR and 87.69% PAP and is the only method to complete the full task. Standalone τ_0 -VLA reaches 76.92% PAP, so hierarchical execution adds 10.77 points; $\pi_{0.5}$, GR00T, and LingBot-VLA reach 54.23%, 26.15%, and 13.08%, respectively. Their failures occur at progressively earlier points in the cooking chain, including heating, oil addition, seasoning, final plating, and serving. These results show the value of preserving prerequisite order through the transition from ingredient preparation to cooking and serving.

C. Multi-Embodiment Evaluation

We evaluate the shared low-level VLA on Collect Laundry with ARX AC One and Tidy Makeup Table with Franka. Compared with the long-horizon AGIBOT G1 tasks, these tasks are structurally simpler and contain only two to five steps. They therefore do not require high-level task decomposition, execution memory, or test-time search. Each method directly executes the full task instruction, and our high-level planner is not used in this evaluation. Table II reports the results.

TABLE I
LONG-HORIZON TASK PERFORMANCE ON AGIBOT G1. EACH TASK USES 10 TRIALS. SR REPORTS SUCCESSFUL TRIALS AS $x/10$, AND PAP DENOTES PREREQUISITE-AWARE PROGRESS. AVG. IS THE UNWEIGHTED MEAN OF THE FOUR TASK-LEVEL RATES.

Method	Clean Room		Ingredient Prep		Tomato and Egg Stir Fry		Make Milk Tea		Avg.	
	SR $\uparrow$	PAP $\uparrow$	SR $\uparrow$	PAP $\uparrow$	SR $\uparrow$	PAP $\uparrow$	SR $\uparrow$	PAP $\uparrow$	SR $\uparrow$	PAP $\uparrow$
GR00T N1.7 [24]	0/10	61.90%	1/10	56.94%	0/10	26.15%	0/10	28.46%	2.50%	43.36%
LingBot-VLA [2]	0/10	67.14%	0/10	27.22%	0/10	13.08%	0/10	63.85%	0.00%	42.82%
$\pi_{0.5}$ [27]	4/10	87.14%	2/10	72.50%	0/10	54.23%	1/10	79.23%	17.50%	73.28%
τ_0 -VLA	4/10	91.43%	4/10	68.33%	0/10	76.92%	7/10	97.69%	37.50%	83.59%
τ_0 -VLA w/ Hierarchical System	8/10	97.86%	8/10	95.00%	4/10	87.69%	5/10	91.42%	62.50%	92.99%

TABLE II
DIRECT-EXECUTION PERFORMANCE ON SHORT-HORIZON MULTI-EMBODIMENT TASKS. COLLECT LAUNDRY IS EVALUATED ON ARX AC ONE, AND TIDY MAKEUP TABLE IS EVALUATED ON FRANKA AS THREE INDEPENDENTLY SCORED INSTRUCTION-FOLLOWING GROUPS. ALL METHODS EXECUTE THE FULL TASK INSTRUCTION WITHOUT A HIGH-LEVEL PLANNER. EACH TASK OR TASK GROUP IS EVALUATED OVER 10 TRIALS. SR REPORTS SUCCESSFUL TRIALS AS $x/10$, AND PAP DENOTES PREREQUISITE-AWARE PROGRESS.

Method	Collect Laundry (ARX)		Tidy Makeup Table (Franka)					
	T-shirt		Cotton Pad		Eyelash Curler		Makeup Puff	
	SR $\uparrow$	PAP $\uparrow$	SR $\uparrow$	PAP $\uparrow$	SR $\uparrow$	PAP $\uparrow$	SR $\uparrow$	PAP $\uparrow$
GR00T N1.7 [24]	4/10	76.00%	10/10	87.50%	8/10	77.50%	7/10	52.50%
LingBot-VLA [2]	2/10	35.00%	9/10	67.50%	3/10	22.50%	3/10	33.75%
$\pi_{0.5}$ [27]	9/10	88.00%	9/10	85.00%	8/10	85.00%	7/10	73.75%
τ_0 -VLA	10/10	97.00%	10/10	95.00%	9/10	92.50%	10/10	95.00%

Collect Laundry results. On Collect Laundry, τ_0 -VLA achieves 10/10 SR and 97.00% PAP, followed by $\pi_{0.5}$ with 9/10 SR and 88.00% PAP. GR00T reaches 4/10 SR and 76.00% PAP, while LingBot-VLA reaches 2/10 SR and 35.00% PAP. LingBot-VLA fails to pick up the garment multiple times. GR00T collides with the foot of the bed four times, terminating those rollouts.

Tidy Makeup Table results. Tidy Makeup Table evaluates instruction-following generalization on Franka. Its three task groups begin from matched visual states, but the language instruction changes the correct object, arm usage, action order, and destination. The observation alone is therefore insufficient to determine the action sequence: the policy must follow the instruction rather than replay a fixed visually conditioned behavior. Cotton Pad and Eyelash Curler test instruction-conditioned object selection and placement, while Makeup Puff additionally requires a longer bimanual drawer-manipulation sequence. Table II reports the three groups separately. τ_0 -VLA achieves 10/10, 9/10, and 10/10 SR on Cotton Pad, Eyelash Curler, and Makeup Puff, respectively, with corresponding PAP scores of 95.00%, 92.50%, and 95.00%. It obtains the highest PAP on all three groups and the highest or tied-highest SR on each group. GR00T shows reasonable instruction following overall, but exhibits pauses during drawer opening and closing and selects the makeup puff instead of the eyelash curler in two trials. LingBot-VLA usually follows the specified arm but appears to select visually nearby objects rather than the instructed target; it reaches only 3/10 SR on both Eyelash Curler and Makeup Puff. $\pi_{0.5}$ is competitive but repeatedly releases and re-grasps the makeup puff, closes the

drawer less smoothly, and omits the cotton pad in one trial. The retries lower PAP even when the final task succeeds, while the omission reduces both PAP and SR.

D. Test-Time Computation Evaluation

We conduct several experiments to quantify the gains in subtask-prediction accuracy achieved by test-time computation (TTC) and assess whether these gains lead to higher success rates in closed-loop real-robot evaluation. Our TTC experiments cover four evaluation settings: Make Milk Tea, in-domain Book Organization, out-of-domain Book Organization, and Clean Room. Make Milk Tea and Clean Room follow the task settings introduced above. Book Organization requires the robot to use its two arms to reorder four books of different heights across four slots, from tallest to shortest. Each time, the robot can exchange the positions of any two books, and the task is completed when all four books are arranged in the correct order. We evaluate both in-domain and out-of-domain (OOD) initial book arrangements: the former appears in the training data, whereas the latter does not. The OOD setting evaluates the high-level planner’s performance when planning from unseen initial observations. More details of the task settings are provided in Appendix B-A. We first measure next-subtask prediction accuracy in an open-loop setting, then evaluate TTC in closed-loop real-robot execution, and finally examine the relationship between computational cost and prediction accuracy.

Open-loop Subtask Prediction. Following the high-level planner protocol above, we evaluate next-subtask prediction at different stages of each task. We compare TTC against

two baselines: (1) **Plan Once**, which directly predicts the next subtask; and (2) **Best-of- N** , which samples N candidates, predicts the next observation for each candidate using the same world model as TTC, and selects the candidate assigned the highest score by the same value model.

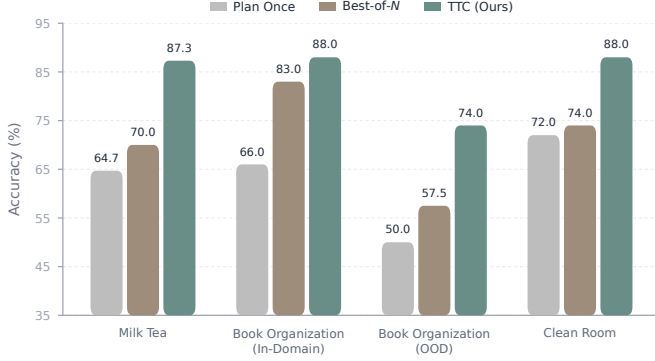


Fig. 4. **LLM-as-a-judge accuracy of test-time computation methods.** Plan Once, Best-of- N , and TTC (Ours) are evaluated on Milk Tea, Book Organization (In-Domain), Book Organization (OOD), and Clean Room.

As shown in Figure 4, TTC achieves the highest accuracy across all four evaluation settings, demonstrating its effectiveness in improving next-subtask selection.

Closed-loop Real-robot Evaluation. We further assess whether gains in subtask-prediction accuracy lead to higher task success rates in closed-loop real-robot evaluation. We deploy the high-level planner with and without TTC while keeping the low-level policy fixed. As reported in Table III, TTC increases the task success rate on every evaluated task.

TABLE III
PERFORMANCE OF TEST-TIME COMPUTATION ON LONG-HORIZON MANIPULATION TASKS. SR DENOTES TASK SUCCESS RATE, AND PAP DENOTES PREREQUISITE-AWARE PROGRESS.

Method	Make Milk Tea		Book Organization		Clean Room	
	SR $\uparrow$	PAP $\uparrow$	SR $\uparrow$	PAP $\uparrow$	SR $\uparrow$	PAP $\uparrow$
Plan Once	5/10	91.54%	6/10	66.67%	5/10	95.00%
TTC	7/10	95.39%	9/10	93.33%	7/10	97.50%

Compute–Accuracy Trade-off. Finally, we examine the relationship between test-time computational cost and subtask-prediction accuracy. Figure 5 reports this relationship on Make Milk Tea and Book Organization. Each point represents an experimental configuration with a different amount of computation, and the orange dashed curves provide approximate saturation fits to the observations. Accuracy rises rapidly when computation is increased from a small budget, showing that additional search substantially improves subtask prediction in the low-compute regime. The marginal gain then gradually decreases and the fitted curves approach a plateau. This trend indicates that TTC provides a favorable compute–accuracy trade-off at moderate budgets, while its benefit eventually saturates as additional computation is allocated.

Effect of additional computation. Relative to plan-once inference, hierarchical TTC improves closed-loop success by 30 percentage points on Make Milk Tea, 30 points on Book Organization, and 20 points on Clean Room. The scaling experiment further measures how these gains change with the candidate and search-depth budgets.

E. High-Level Planner Analysis

Following the high-level planner protocol, we evaluate the contribution of execution memory while holding the annotated samples and output interface fixed. Table IV reports next-subtask prediction accuracy and inference time with and without memory.

Memory ablation result. Execution memory improves next-subtask accuracy from 43.9% to 54.9%, an increase of 11.0 percentage points, while mean inference time increases from 0.89 s to 2.17 s.

Instruction steerability. The planner accepts an ordered L2 stage plan as part of its native input, so a user can express a preferred ordering for an otherwise ambiguous task without retraining. This is the same pathway exercised by our instruction-following generalization evaluation (Tidy Makeup Table, Table II): from matched visual states, the correct object, acting arm, action order, and destination are determined by the language input rather than by a fixed visually conditioned behavior. The planner re-decomposes the task from the provided stage plan and the current observation, so steerability is a property of the existing high-level planner rather than a separately tuned capability.

F. Failure and Recovery Analysis

We separate repeated or skipped subtasks, incorrect object or scene-state estimates, failed recovery, unsafe or invalid subtask generation, and low-level execution failures. The high-level planner accounts for [XX]% of remaining failures, while low-level execution accounts for [XX]%. Recovery succeeds in [XX]% of recoverable failures, with [FAILURE MODE] as the dominant unresolved failure mode.

VI. CONCLUSION

We presented τ_0 -VLA, a hierarchical VLA system for long-horizon mobile manipulation that combines a memory-augmented High-Level Planner with a low-level whole-body policy. The planner handles routine decisions efficiently and allocates world-model-guided computation when a decision benefits from explicit consequence evaluation. The low-level policy is trained from heterogeneous robot and vision-language data for deployable whole-body mobile manipulation. Together, the two levels provide a practical way to allocate reasoning over long task horizons while retaining stable real-robot execution.

REFERENCES

- [1] AGIBOT Innovation. AGIBOT G1: Universal embodied intelligent robot. <https://www.agibot.com/products/G1>, 2026. Product page, accessed July 1, 2026.

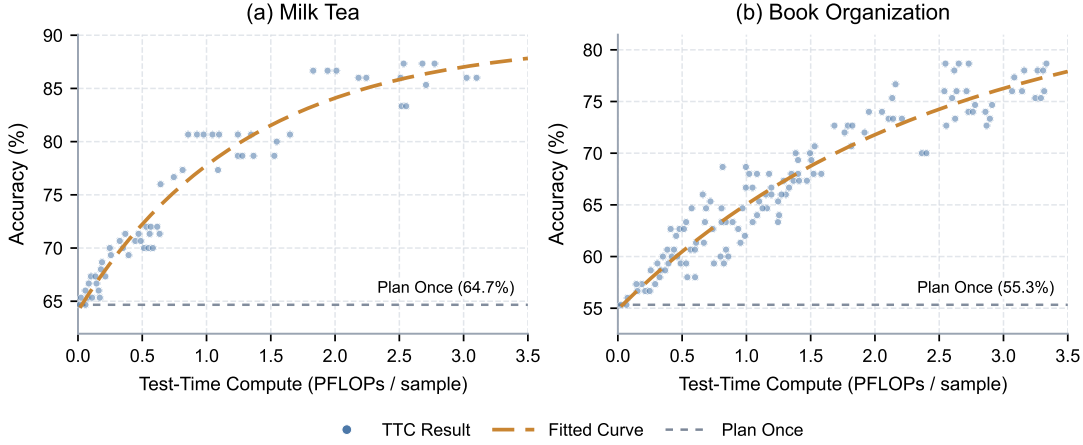


Fig. 5. **Compute-accuracy trade-off.** Accuracy improves as additional test-time compute is allocated on (a) Milk Tea and (b) Book Organization. Each point denotes an experimental setting, the orange dashed curves show saturation fits, and the gray dashed lines indicate the Plan Once baselines.

TABLE IV

SUBTASK PREDICTION ACCURACY AND INFERENCE TIME WITH AND WITHOUT MEMORY AS INPUT. RESULTS ARE REPORTED AS MEAN $\pm$ STANDARD DEVIATION.

Method	With Memory		Without Memory	
	Subtask Acc. (%) $\uparrow$	Time (s) $\downarrow$	Subtask Acc. (%) $\uparrow$	Time (s) $\downarrow$
High-Level Planner	54.9	2.17 ± 0.34	43.9	0.89 ± 0.20
Qwen 3.5	9.8	1.95 ± 0.24	2.3	3.38 ± 1.41
RoboBrain 2.5	14.3	1.54 ± 0.73	1.1	0.74 ± 1.09
Molmo2 ER	1.1	1.08 ± 0.13	0.0	0.46 ± 0.06
ChatGPT 5.5	20.7	10.65 ± 5.09	1.5	9.30 ± 2.86
Claude Opus 4.8	17.4	7.22 ± 5.78	3.0	7.98 ± 8.16

- [2] Ant Group. LingBot-VLA: A pragmatic vla foundation model, 2026.
- [3] ARX Robotics. ROS2 AC-one Play: Imitation learning sdk. https://github.com/ARXroboticsX/ROS2_AC-one_Play, 2026. Accessed July 1, 2026.
- [4] Suneel Belkale, Tianli Ding, Ted Xiao, Pierre Sermanet, Quon Vuong, Jonathan Tompson, Yevgen Chebotar, Debiddatta Dwibedi, and Dorsa Sadigh. RT-H: Action hierarchies using language. *arXiv preprint arXiv:2403.01823*, 2024.
- [5] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [6] Kevin Black, Mitsuhiko Nakamoto, Pranav Atreya, Homer Walke, Chelsea Finn, Aviral Kumar, and Sergey Levine. Zero-shot robotic manipulation with pretrained image-editing diffusion models. In *International Conference on Learning Representations*, volume 2024, pages 33431–33452, 2024.
- [7] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. RT-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [8] Jun Cen, Chaohui Yu, Hangjie Yuan, Yuming Jiang, Siteng Huang, Jiayan Guo, Xin Li, Yibing Song, Hao Luo, Fan Wang, et al. WorldVLA: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.
- [9] Chi-Lam Cheang, Guangzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, et al. GR-2: A generative video-language-action model with web-scale knowledge for robot manipulation. *arXiv preprint arXiv:2410.06158*, 2024.
- [10] Yilun Du, Sherry Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Josh Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning universal policies via text-guided video generation. *Advances in Neural Information Processing Systems*, 36:9156–9172, 2023.
- [11] Yilun Du, Sherry Yang, Pete Florence, Fei Xia, Ayzaan Wahid, Pierre Sermanet, Tianhe Yu, Pieter Abbeel, Joshua B Tenenbaum, Leslie Kaelbling, et al. Video language planning. In *International Conference on Learning Representations*, volume 2024, pages 31138–31155, 2024.
- [12] Yunhai Feng, Jiaming Han, Zhuoran Yang, Xiangyu Yue, Sergey Levine, and Jianlan Luo. Reflective planning:

- Vision-language models for multi-stage long-horizon robotic manipulation. *arXiv preprint arXiv:2502.16707*, 2025.
- [13] Figure AI. Helix: A vision-language-action model for generalist humanoid control. <https://www.figure.ai/news/helix>, 2025.
- [14] Franka Robotics. Franka Research 3. <https://franka.de/franka-research-3>, 2026. Product page, accessed July 1, 2026.
- [15] GenRobot. GenRobot 10Kh RealOmni-Open: A large-scale real-world embodied manipulation dataset. <https://huggingface.co/datasets/genrobot2025/10Kh-RealOmin-OpenData>, 2026. GenRobot AI open dataset.
- [16] Google DeepMind. Gemini Robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer, 2025.
- [17] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
- [18] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [19] Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning*, pages 287–318. PMLR, 2023.
- [20] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Open-VLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [21] Jacky Kwok, Christopher Agia, Rohan Sinha, Matt Foutter, Shulu Li, Ion Stoica, Azalia Mirhoseini, and Marco Pavone. RoboMonkey: Scaling test-time sampling and verification for vision-language-action models. *arXiv preprint arXiv:2506.17811*, 2025.
- [22] Shiyu Liu, Yucheng Han, Peng Xing, Fukun Yin, Rui Wang, Wei Cheng, Jiaqi Liao, Yingming Wang, Honghao Fu, Chunrui Han, et al. Step1X-Edit: A practical framework for general image editing. *arXiv preprint arXiv:2504.17761*, 2025.
- [23] Mitsuhiko Nakamoto, Oier Mees, Aviral Kumar, and Sergey Levine. Steering your generalists: Improving robotic foundation models via value guidance. *arXiv preprint arXiv:2410.13816*, 2024.
- [24] NVIDIA. Isaac GR00T N1.7: An open reasoning vision-language-action model for humanoid robots. <https://huggingface.co/blog/nvidia/gr00t-n1-7>, 2026. NVIDIA Isaac GR00T N1.7; foundational report: GR00T N1, arXiv:2503.14734.
- [25] Octo Model Team. Octo: An open-source generalist robot policy, 2024.
- [26] Physical Intelligence. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better, 2025.
- [27] Physical Intelligence. $\pi_{0.5}$: A vision-language-action model with open-world generalization, 2025.
- [28] Physical Intelligence. $\pi_{0.7}$: A steerable generalist robotic foundation model with emergent capabilities, 2026.
- [29] Qwen Team. Qwen3.5. https://huggingface.co/docs/transformers/model_doc/qwen3_5, 2026. Qwen3.5 model family.
- [30] RoboCOIN Team. RoboCOIN: An open-sourced bimanual robotic data collection for integrated manipulation, 2025. URL <https://arxiv.org/abs/2511.17441>.
- [31] RoboMIND Team. RoboMIND: Benchmark on multi-embodiment intelligence normative data for robot manipulation, 2024. URL <https://arxiv.org/abs/2412.13877>.
- [32] RoVer Authors. RoVer: Robot reward model as test-time verifier for vision-language-action models, 2025.
- [33] Lucy Xiaoyang Shi, Brian Ichter, Michael Equi, Liyiming Ke, Karl Pertsch, Quan Vuong, James Tanner, Anna Walling, Haohuan Wang, Niccolo Fusai, et al. Hi Robot: Open-ended instruction following with hierarchical vision-language-action models. *arXiv preprint arXiv:2502.19417*, 2025.
- [34] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025.
- [35] Tencent Hunyuan. HY-Embodied-0.5: Embodied foundation models for real-world agents, 2026. URL <https://github.com/Tencent-Hunyuan/Hy-Embodied-0.5-VLA>.
- [36] X-VLA Team. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model, 2025.
- [37] XDOF. ABC-130K: Scalable behavior cloning with open data, training, and evaluation. <https://abc.bot/>, 2026. Open-source bimanual robot manipulation dataset.
- [38] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.

APPENDIX A ADDITIONAL DETAILS

This appendix provides additional details on the robot platforms, low-level training data, and High-Level Planner data construction.

A. Robot Platform Details

AGIBOT G1. AGIBOT G1 [1] is a wheeled humanoid with an omnidirectional four-wheel-steering base, dual 7-DoF arms, and modular end-effectors; our setup uses parallel-jaw grippers. Its sensing suite includes head-mounted RGB-D and fisheye cameras and a wrist-mounted camera on each arm.

ARX AC One. ARX AC One [3] is a bimanual platform with dual 6-DoF X5 arms and parallel-jaw grippers. We use wrist-mounted cameras together with a fixed central camera.

Franka Research 3. Our bimanual Franka Research 3 [14] setup comprises two 7-DoF torque-controlled arms with custom 3D-printed grippers and three RGB cameras. Across all three embodiments, the policy receives multi-view RGB images and proprioceptive state, and the native robot commands are represented in the unified 40-dimensional action layout defined in Section III-D.

B. Low-Level Data Details

The low-level training corpus combines approximately 32.7K hours of internally collected demonstrations with public robot datasets. The internal data comprise 21.9K hours on AGIBOT G1, 9.25K hours on UMI, 585 hours on AGIBOT G2, 578 hours on ARX AC One, and 347 hours on Franka. The public mixture includes RoboMIND [31], RoboCOIN [30], GenRobot 10Kh RealOmni-Open [15], Hy-Embodied-0.5-VLA-Data [35], and ABC-130K [37], expanding the coverage of embodiments, tasks, and contact-rich manipulation skills.

We additionally use multimodal co-training data for image, document, and video instruction following; visual grounding and dense description; spatial and depth reasoning; and robot-centric perception, including affordance, progress, failure, recovery, trajectory, and multi-view understanding. This mixture contains hundreds of curated dataset entries together with an additional five-million-example corpus. Sampling and preprocessing follow the same co-training pipeline throughout low-level pretraining.

C. High-Level Planner Data Construction

Planner supervision is generated automatically from task instructions, stage descriptions, executable subtask annotations, segmented demonstrations, and videos. In the data pipeline, we denote these three instruction levels as L1, L2, and L3, respectively. Construction proceeds in three steps. **(1) Pre-labeling.** Using google/Gemma4-31B-it as the annotator, we generate a `<think>` field that summarizes scene state, progress, constraints, and failures, together with a `<memory>` field that compresses completed stages while retaining detail for the active subtask. The `<subtask>` field reuses the existing executable-subtask annotation. **(2) Keyframe extraction.** Following each subtask’s temporal range, we extract three synchronized views (`top_head`, `hand_left`, `hand_right`) with `ffmpeg`. **(3) VQA assembly.** We construct supervision for full planner prediction, task decomposition, error detection, structured reasoning, and next-subtask prediction. Pretraining primarily uses the full planner target and next-subtask prediction.

To make the High-Level Planner robust to deployment-time memory misalignment, we perturb *only* the input memory while reading the corrected target from the demonstration. This procedure derives the five instance families in Table V without additional annotation. Here $\mathcal{M}_N$ denotes the memory

upon entering segment N ; *within-subtask* is the aligned case, and the remaining families target distinct failure modes.

For *error-think*, the `<think>` field first flags the failure and then repairs memory according to the failure type. Recoverable failures, such as an empty grasp, are retried with memory unchanged; failures that undo prior progress, such as a dropped object, trigger rollback to the preceding subtask. Samples marked `restorable=false` are skipped, and *rollback* instances are capped at 10–15% to avoid teaching the planner to distrust correct memory. We additionally apply six-dimensional visually grounded augmentation to the L1/L2 task and stage instructions to improve instruction diversity and zero-shot steerability.

Quality control. Quality control operated in two phases. During prompt development, fixed-seed stratified human spot-checks on small batches surfaced failure modes—chiefly cross-task contamination, where the annotator hallucinated action constraints from an unrelated task (e.g. milk-tea constraints appearing under a fruit-shelving task). All visual prompts embed a verification rule that requires omitting any attribute that cannot be confirmed from the images (“when in doubt, leave it out”), and each identified failure mode was hardened into a programmatic filter applied at scale: a cross-task contamination filter, an empty-`<subtask>` detector, and a memory-contamination filter. Once the prompts and filters were fixed, the full corpus was pre-labeled and filtered programmatically, without further per-sample human inspection.

The contamination filter builds a per-episode noun vocabulary from the L1 and L3 annotations and rejects a constraint whose operated object lies entirely outside this vocabulary, or whose slot nouns overlap it by less than 20%. Tightening the criterion from a naive disjointness test (discard rate 0.33%, with residual contamination) through an over-aggressive ratio threshold (discard rate 2.32%, which removed many valid but verbose constraints) to the final two-rule filter with an expanded stop-word list (discard rate 0.42%) left zero residual contamination on a 3.26M-sample validation set. Structural dirty-data filtering additionally removes empty-`<subtask>` episodes and the downstream memory corruption they induce, using a three-tier policy by per-task dirty ratio (tasks above 90% dirty are excluded outright, tasks between 10% and 90% are filtered at the episode level, and tasks below 10% have only their empty-`<subtask>` samples removed); this discards 11.74% of episodes and retains 40.4M (88.26%) clean samples. The held-out planner evaluation set is drawn from frames unseen in training.

D. Asynchronous Dual-System Serving

Since a high-level decision is far slower than one low-level control period, we serve the two systems asynchronously so that planner latency never stalls control. A background worker continuously recomputes the next subtask for each active episode and publishes the committed z_k^* to a per-episode cache (refreshed about every 1 s). The control loop only reads this cache: each tick takes the cached subtask as the current command $c_t = z_k^*$ and returns an action chunk immediately,

TABLE V

HIGH-LEVEL INSTANCE FAMILIES, SYNTHESIZED BY PERTURBING ONLY THE INPUT MEMORY WHILE READING THE CORRECTED TARGET FROM THE DEMONSTRATION. $\mathcal{M}_N$ IS THE MEMORY UPON ENTERING SEGMENT N . A SINGLE UNIFIED `<THINK>/<MEMORY>/<SUBTASK>` FORMAT INSTANTIATES ALL OF THEM AT ZERO EXTRA ANNOTATION.

Family	Sampling position	Input $\rightarrow$ target memory	Target subtask	Deployment failure countered	Mix
<i>within-subtask</i>	anywhere in seg. N	$\mathcal{M}_N \rightarrow \mathcal{M}_N$	seg. N	— (aligned, normal progression)	58%
<i>transition</i>	tail of seg. N	$\mathcal{M}_N \rightarrow \mathcal{M}_{N+1}$	seg. $N+1$	knowing when to switch stage	15%
<i>catch-up</i>	head of seg. N	$\mathcal{M}_{N-1} \rightarrow \mathcal{M}_N$	seg. N	memory lag (behind the visual state)	10%
<i>rollback</i>	late in seg. N	$\mathcal{M}_{N+1\dots N+3} \rightarrow \mathcal{M}_N$	retry seg. N	memory run-ahead (over-optimistic)	12%
<i>error-think</i>	annotated failure frame	$\mathcal{M}_N \rightarrow$ type-dependent	recovery step	unnoticed execution failure	5%

without waiting for the planner. A slow or failed high-level decision thus only delays the next command refresh, while π_θ keeps executing the current z_k^* at the control rate (~ 30 Hz).

APPENDIX B

TEST-TIME COMPUTATION EVALUATION DETAILS

A. Book Organization Task Settings

The Book Organization task contains four books of different heights and four fixed slots. At the beginning of each trial, the four books are placed in a shuffled order. At each step, the robot can use its two arms to exchange the positions of any two books. The task is completed when all four books are arranged from tallest to shortest. The high-level planner must therefore predict which pair of books should be exchanged next to make progress toward the target ordering. In the in-domain setting, the initial arrangement of the four books has appeared in the training data. In the out-of-domain (OOD) setting, the initial arrangement of the four books has not appeared in the training data. The OOD setting is designed to evaluate the robustness of the high-level planner to observations arising from initial book arrangements not encountered during training.

B. LLM-as-a-Judge Protocol

For the open-loop TTC evaluation, each sample contains a robot observation from a particular stage of a task and the corresponding ground-truth next subtask. Each method predicts the next subtask from the same input, and an LLM judge evaluates whether the prediction is correct with respect to the ground truth.